CSCE 5222 — Feature Engineering for Time-Series

Group 18 — Code Documentation

Ashish Rathnakar Shetty (ID: 11808466) · Kushal Sai Venigalla (ID: 11852559)
Department of Computer Science and Engineering, University of North Texas

---

1. Project Overview

This document explains every cell and every important line of code in our `Group18_Demo.ipynb` Google Colab notebook. The notebook implements our complete feature engineering pipeline across three time-series tasks: classification, forecasting, and anomaly detection.

1.1 What the Notebook Does

- Implements all 22 Catch22 features from scratch in pure NumPy
- Tests four feature engineering strategies on five UCR datasets
- Builds lag and rolling features for ETTh1 forecasting
- Trains a self-supervised MLP for embedding learning
- Uses SHAP values as an active feature selection mechanism

1.2 How to Run

1. Open the notebook in Google Colab
2. Click **Runtime → Run all**
3. Wait approximately 2–3 minutes for everything to download and execute

---

2. Cell 1 — Install and Import

2.1 Purpose

Installs the only missing dependency (`shap`) and imports every library used in the rest of the notebook.

2.2 Line-by-Line Explanation

```
!pip install shap --quiet
```

Installs the SHAP library using pip. The `!` prefix tells Colab to run this as a shell command. The `--quiet` flag suppresses the verbose installation output.

```
import os, warnings, urllib.request, zipfile, glob
```

Imports five standard library modules: `os` for file paths, `warnings` to suppress deprecation messages, `urllib.request` for downloading datasets, `zipfile` for extracting downloaded archives, and `glob` for finding files matching a pattern.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import seaborn as sns
```

Imports the data science stack. NumPy for numerical operations, pandas for tabular data, matplotlib for plotting, `mpatches` for legend patches, and seaborn for statistical visualization styling.

```
from sklearn.preprocessing import StandardScaler, LabelEncoder
```

Imports two preprocessing utilities. `StandardScaler` standardizes features to zero mean and unit variance. `LabelEncoder` converts string class labels into integer indices.

```
from sklearn.ensemble import (GradientBoostingClassifier,
                              GradientBoostingRegressor,
                              IsolationForest)
```

Imports three ensemble models. `GradientBoostingClassifier` and `GradientBoostingRegressor` are our main models for classification and forecasting. `IsolationForest` is an unsupervised anomaly detection algorithm.

```
from sklearn.linear_model import Ridge
from sklearn.decomposition import PCA
from sklearn.neural_network import MLPRegressor
```

Imports `Ridge` regression for the forecasting baseline, `PCA` for visualizing high-dimensional features in 2D, and `MLPRegressor` for the self-supervised embedding network.

```
from sklearn.metrics import (accuracy_score, f1_score,
                             mean_squared_error, mean_absolute_error,
                             roc_auc_score)
```

Imports all evaluation metrics. Accuracy and F1 for classification, MSE and MAE for forecasting, ROC-AUC for anomaly detection.

```
import shap
```

Imports the SHAP library, which we use to compute feature importance values for our SHAP-guided pruning approach.

```
warnings.filterwarnings('ignore')
np.random.seed(42)
```

Suppresses warnings to keep the output clean. Sets the NumPy random seed to 42 so all results are reproducible across runs.

```
plt.rcParams.update({
    'font.size': 11,
    'figure.dpi': 110,
    'axes.spines.top': False,
    'axes.spines.right': False,
})
```

Configures matplotlib defaults. Sets a readable font size, increases the resolution to 110 DPI, and removes the top and right border lines for a cleaner appearance.

```
print("Everything imported successfully!")
print(f"  numpy   {np.__version__}")
print(f"  pandas  {pd.__version__}")
print(f"  sklearn {__import__('sklearn').__version__}")
print(f"  shap    {shap.__version__}")
```

Confirms all imports worked and prints the version of each major library, which helps with reproducibility and debugging.

---

3. Cell 2 — The Catch22 Implementation

3.1 Purpose

Defines our pure-Python implementation of all 22 Catch22 features. The original `pycatch22` library requires a C compiler to install, which causes problems on many platforms. Our NumPy-only version works anywhere Python runs.

3.2 Function Signature

```
def catch22_features(x):
    """Compute all 22 Catch22 features for a 1D time series."""
    x = np.asarray(x, dtype=float)
    n = len(x)
    feats = {}
```

Takes a 1D array `x`, converts it to a NumPy float array, stores its length `n`, and creates an empty dictionary to hold the 22 features.

3.3 Distribution Features (6 features)

```
mu = np.mean(x)
s  = np.std(x, ddof=1) + 1e-10
```

Computes the mean and standard deviation of the signal. The `ddof=1` flag uses the sample standard deviation formula (divides by n-1 instead of n). The tiny `1e-10` added to the standard deviation prevents division-by-zero errors later.

```
feats['mean']         = float(mu)
feats['std']          = float(s)
feats['skewness']     = float(np.mean(((x - mu) / s) ** 3))
feats['kurtosis']     = float(np.mean(((x - mu) / s) ** 4) - 3.0)
feats['above_mean']   = float(np.mean(x > mu))
feats['outlier_frac'] = float(np.mean(np.abs((x - mu) / s) > 2))
```

- `mean` and `std` are stored directly.
- `skewness` measures asymmetry: positive means a longer tail to the right.
- `kurtosis` measures tail heaviness: subtract 3 to make a Gaussian have kurtosis zero.
- `above_mean` is the fraction of values above the mean.
- `outlier_frac` is the fraction of values more than 2 standard deviations from the mean.

3.4 Temporal Dependence Features (4 features)

```
xn  = x - mu
acf = np.correlate(xn, xn, mode='full')[n-1:]
acf_norm = acf / (acf[0] + 1e-10)
```

Computes the autocorrelation function. `xn` is the mean-centered signal. `np.correlate` with `mode='full'` returns the full cross-correlation, and we take the second half (positive lags only). Then we normalize so `acf[0] = 1`.

```
cz  = np.where(np.diff(np.sign(acf_norm)))[0]
c1e = np.where(acf_norm < 1.0/np.e)[0]
feats['first_zero_acf'] = int(cz[0])  if len(cz)  else n
feats['first_1e_acf']   = int(c1e[0]) if len(c1e) else n
```

- `cz` finds indices where the autocorrelation crosses zero (sign change).
- `c1e` finds indices where the autocorrelation drops below 1/e (a standard decay threshold).
- We store the first such index, or `n` if it never happens.

```
feats['acf_lag1'] = float(np.corrcoef(x[:-1], x[1:])[0,1]) if n > 1 else 0.0
feats['acf_lag2'] = float(np.corrcoef(x[:-2], x[2:])[0,1]) if n > 2 else 0.0
```

Computes Pearson correlation between the signal and itself shifted by 1 step (`acf_lag1`) and 2 steps (`acf_lag2`).

3.5 Complexity and Entropy Features (4 features)

```
counts, _ = np.histogram(x, bins=10)
p = counts / (counts.sum() + 1e-10)
feats['hist_entropy'] = float(-np.sum(p * np.log(p + 1e-10)))
```

Histogram entropy. Bin the values into 10 bins, normalize to a probability distribution, then compute Shannon entropy `-Σ p log p`.

```
if n >= 3:
    pats = {}
    for i in range(n - 2):
        pat = tuple(np.argsort(x[i:i+3]))
        pats[pat] = pats.get(pat, 0) + 1
    tot = sum(pats.values())
    pp  = np.array(list(pats.values())) / tot
    feats['perm_entropy'] = float(-np.sum(pp * np.log(pp + 1e-10)))
else:
    feats['perm_entropy'] = 0.0
```

Permutation entropy. For each window of 3 consecutive values, the rank ordering is one of 6 possible patterns. We count how often each pattern occurs, normalize, and compute Shannon entropy of those counts.

```
med    = np.median(x)
binary = ''.join('1' if v >= med else '0' for v in x)
i2, k, lz = 0, 1, 1
while k + lz <= n:
    if binary[i2:i2+lz] not in binary[:k]:
        lz += 1
    else:
        i2 += 1
        if i2 + lz > k:
            k += lz; i2, lz = 0, 1
feats['lempel_ziv'] = k / n if n > 0 else 0.0
```

Lempel-Ziv complexity. Convert the signal to a binary string based on whether each value is above or below the median. Then walk through the string, looking for new substrings that have not appeared before. The complexity is the count of distinct substrings divided by `n`.

```
fft_v = np.abs(np.fft.rfft(x)) ** 2
fft_v = fft_v / (fft_v.sum() + 1e-10)
feats['spectral_entropy'] = float(-np.sum(fft_v * np.log(fft_v + 1e-10)))
```

Spectral entropy. Compute the power spectrum via FFT, normalize to a probability distribution, then compute Shannon entropy. High spectral entropy means power is

spread evenly across frequencies; low entropy means power is concentrated at a few frequencies.

3.6 Frequency and Scaling Features (3 features)

```
freqs = np.fft.rfftfreq(n)
feats['peak_freq']   = float(freqs[np.argmax(fft_v)]) if len(fft_v) else 0.0
mid = len(fft_v) // 2
feats['hi_lo_ratio'] = float((fft_v[mid:].sum()+1e-10) /
(fft_v[:mid].sum()+1e-10))
```

- `peak_freq` is the frequency where the power spectrum is highest.
- `hi_lo_ratio` is the ratio of total power in the high-frequency half to the low-frequency half.

```
if n > 20:
    half = n // 2
    rs = []
    for seg in [x[:half], x[half:]]:
        seg = seg - seg.mean()
        cs  = np.cumsum(seg)
        rs.append((cs.max()-cs.min())/(np.std(seg, ddof=1)+1e-10))
    feats['hurst'] = float(np.log(np.mean(rs)+1e-10)/np.log(half+1e-10))
else:
    feats['hurst'] = 0.5
```

Hurst exponent via R/S analysis. Split the signal in half, compute the rescaled range (max minus min of cumulative sum, divided by std) for each half, then compute the Hurst exponent from the log ratio. A Hurst near 0.5 means random walk; above 0.5 means trending; below 0.5 means mean-reverting.

3.7 Dynamics Features (5 features)

```
diffs = np.diff(x)
feats['diff_variance']      = float(np.var(diffs, ddof=1)) if len(diffs)>1
else 0.0
feats['n_zero_crossings']   = int(np.sum(np.diff(np.sign(xn)) != 0))
feats['time_reversibility'] = float(np.mean(diffs**3))
feats['nonlinearity']       = float(abs(np.corrcoef(x[:-1],(x**2)[1:])
[0,1])) if n>3 else 0.0
streaks = np.split(xn, np.where(xn <= 0)[0])
feats['longest_above_mean'] = int(max((len(sg) for sg in streaks if
np.all(sg>0)), default=0))
```

- `diff_variance` measures signal roughness via variance of first differences.
- `n_zero_crossings` counts how often the mean-centered signal crosses zero.
- `time_reversibility` is the mean cubed difference, which measures asymmetry in time direction.
- `nonlinearity` is the absolute correlation between `x[t-1]` and `x[t]^2`, capturing nonlinear dependence.
- `longest_above_mean` is the longest consecutive run of values above the mean.

3.8 Helper Function

```
def extract_catch22(X):
    rows  = [list(catch22_features(row).values()) for row in X]
    names = list(catch22_features(X[0]).keys())
    return np.array(rows), names
```

Applies the Catch22 function to every row of a 2D array. Returns a feature matrix and the list of feature names.

---

4. Cell 3 — SHAP-Guided Feature Pruning Helper

4.1 Purpose

Implements our active feature selection mechanism. Trains a proxy model, computes SHAP values, and returns the top 60% of features by mean absolute SHAP importance.

4.2 Function Definition

```
def shap_prune(X_tr, y_tr, X_te, feat_names, keep_frac=0.6, task='clf'):
    if task == 'clf':
        proxy = GradientBoostingClassifier(n_estimators=80, max_depth=3,
random_state=42)
    else:
        proxy = GradientBoostingRegressor(n_estimators=80, max_depth=3,
random_state=42)
    proxy.fit(X_tr, y_tr)
```

Trains a small GradientBoosting proxy model. We use a small model (only 80 trees, depth 3) because we just need it to identify which features matter, not to be the final model. Random state is fixed for reproducibility.

```
    explainer = shap.TreeExplainer(proxy)
    sv = explainer.shap_values(X_tr)
```

Creates a SHAP TreeExplainer specifically optimized for tree-based models. Then computes SHAP values for every training sample. Each SHAP value tells us how much that feature contributed to the prediction for that sample.

```
    if isinstance(sv, list):
        mean_abs = np.mean([np.abs(s).mean(axis=0) for s in sv], axis=0)
    elif isinstance(sv, np.ndarray) and sv.ndim == 3:
        mean_abs = np.abs(sv).mean(axis=(0, 2))
    else:
        mean_abs = np.abs(sv).mean(axis=0)
```

SHAP returns different formats depending on whether the task is binary or multi-class.

- For binary classification, `sv` is a single 2D array.
- For multi-class with old SHAP, `sv` is a list of 2D arrays (one per class).
- For multi-class with new SHAP, `sv` is a single 3D array. We handle all three cases and reduce to a single importance score per feature by averaging the absolute SHAP values across samples (and classes if multi-class).

```
    n_keep  = max(1, int(len(feat_names) * keep_frac))
    top_idx = np.sort(np.argsort(mean_abs)[::-1][:n_keep].astype(int))
    kept    = [feat_names[i] for i in top_idx]
    return X_tr[:, top_idx], X_te[:, top_idx], kept, mean_abs, proxy
```

- Calculate how many features to keep (60% of total).
- `np.argsort(mean_abs)[::-1]` sorts feature indices in descending order of importance.
- Take the top `n_keep` and re-sort them in original order so the column order is preserved.
- Slice both train and test feature matrices to keep only those columns.
- Return the pruned matrices, the kept feature names, the importance scores, and the proxy model.

---

5. Cell 4 — Load All Three Datasets

5.1 Purpose

Downloads the UCR classification datasets and the ETTh1 forecasting dataset, generates the WESAD-like PPG signal, and plots the PPG signal so we can see the anomalies.

5.2 Setup

```
DATA_DIR = '/content/csce5222_data'
os.makedirs(DATA_DIR, exist_ok=True)
```

Creates a folder in Colab's local storage (`/content` is Colab's working directory) to hold all downloaded data. `exist_ok=True` prevents an error if the folder already exists.

5.3 UCR File Parser

```
def parse_ts_file(filepath):
    X, y = [], []
    with open(filepath, 'r') as f:
        in_data = False
        for line in f:
            line = line.strip()
            if line.lower() == '@data':
                in_data = True; continue
            if in_data and line and not line.startswith('#'):
                if ':' in line and ',' in line:
                    data_part, label = line.rsplit(':', 1)
                    vals = [float(v) for v in data_part.split(',')]
                elif ',' in line:
                    parts = line.split(',')
                    label = parts[-1].strip()
                    vals  = [float(v) for v in parts[:-1]]
                else:
                    continue
                X.append(vals); y.append(label.strip())
    le    = LabelEncoder()
    y_enc = le.fit_transform(y)
    min_l = min(len(r) for r in X)
    return np.array([r[:min_l] for r in X]), y_enc
```

UCR files use the `.ts` format with `@data` marker. The function:

1. Reads line by line, ignoring everything until `@data`.

2. Each data line is either `value1,value2,...,valueN:label` (colon separator) or `value1,...,valueN,label` (comma separator). We handle both.
3. Encodes the string labels as integers using `LabelEncoder`.
4. Truncates all rows to the minimum length (some datasets have variable-length rows).

5.4 Downloading UCR Datasets

```
ucr_names = ['GunPoint', 'ECG200', 'ItalyPowerDemand', 'SyntheticControl',
'TwoLeadECG']
ucr_data  = {}

for name in ucr_names:
    local_dir = os.path.join(DATA_DIR, name)
    local_zip = os.path.join(DATA_DIR, f'{name}.zip')
    try:
        if not os.path.exists(local_dir):
            url = f"https://www.timeseriesclassification.com/aeon-
toolkit/{name}.zip"
            urllib.request.urlretrieve(url, local_zip)
            with zipfile.ZipFile(local_zip, 'r') as z:
                z.extractall(local_dir)
        ts_files = glob.glob(os.path.join(local_dir, '**/*.ts'),
recursive=True)
        train_f  = [f for f in ts_files if 'TRAIN' in f.upper()]
        test_f   = [f for f in ts_files if 'TEST'  in f.upper()]
        X_tr, y_tr = parse_ts_file(train_f[0])
        X_te, y_te = parse_ts_file(test_f[0])
        ucr_data[name] = (X_tr, y_tr, X_te, y_te)
    except Exception as e:
        print(f"  {name}: FAILED — {e}")
```

For each UCR dataset:

1. Skip download if already present.
2. Download the ZIP from the official UCR mirror.
3. Extract it.
4. Find the TRAIN and TEST .ts files using glob.
5. Parse them and store in the `ucr_data` dictionary.

5.5 Downloading ETTh1

```
ett_path = os.path.join(DATA_DIR, 'ETTh1.csv')
if not os.path.exists(ett_path):
    urllib.request.urlretrieve(
        "https://raw.githubusercontent.com/zhouhaoyi/ETDataset/main/ETT-small/ETTh1.csv",
        ett_path)
ett_df = pd.read_csv(ett_path, parse_dates=['date'])
```

Downloads the ETTh1 CSV from the official Informer GitHub repo and parses dates automatically.

5.6 Generating the WESAD-like PPG Signal

```
rng    = np.random.RandomState(7)
n_ppg  = 2000
t_ppg  = np.linspace(0, 4 * np.pi, n_ppg)
bvp    = np.sin(t_ppg * 1.2) + 0.3 * np.sin(t_ppg * 3.6) + rng.normal(0, 0.1, n_ppg)
labels = np.zeros(n_ppg, dtype=int)
```

- Use a separate random state (`rng`) seeded with 7 so the PPG signal is reproducible.
- Generate 2000 time points spanning 4π.
- Combine two sinusoids of different frequencies plus Gaussian noise.
- Initialize anomaly labels as all zeros.

```
anomaly_segs = [(300, 340), (700, 730), (1100, 1160), (1500, 1540), (1800,
1840)]
for start, end in anomaly_segs:
    kind = rng.randint(0, 3)
    if kind == 0:
        bvp[start:end] += rng.uniform(3, 5, end - start)        # spike
    elif kind == 1:
        bvp[start:end]  = rng.uniform(-0.05, 0.05, end - start) # flatline
    else:
        bvp[start:end] += rng.normal(0, 2.5, end - start)       # high noise
    labels[start:end] = 1
```

Define five anomaly regions, each randomly assigned one of three artifact types:

- **Spike**: add a large positive value (3 to 5 standard deviations).
- **Flatline**: replace with near-zero values, simulating sensor detachment.
- **High noise**: add high-variance noise, simulating motion artifacts. Mark all samples in those regions as anomalies.

5.7 Plotting the PPG Signal

```
fig, ax = plt.subplots(figsize=(14, 3))
ax.plot(bvp, color='steelblue', lw=0.8)
in_anom, seg_start = False, 0
for i, l in enumerate(labels):
    if l == 1 and not in_anom:
        seg_start = i; in_anom = True
    elif l == 0 and in_anom:
        ax.axvspan(seg_start, i, color='tomato', alpha=0.3)
        in_anom = False
if in_anom:
    ax.axvspan(seg_start, n_ppg, color='tomato', alpha=0.3)
```

Plots the BVP signal in blue, then walks through the labels to find anomaly regions and draws a translucent red overlay on each one using `axvspan`.

---

6. Cell 5 — Run Classification on All 5 UCR Datasets

6.1 Purpose

Iterates through all five UCR datasets and runs three approaches on each: Raw Features, Catch22, and Catch22+SHAP.

6.2 The Main Loop

```
clf_results = []

for ds_name, (X_tr, y_tr, X_te, y_te) in ucr_data.items():
```

Initializes a list to store results. Then iterates through each dataset, unpacking the train/test split.

6.3 Baseline: Raw Features

```
max_len = min(50, X_tr.shape[1])
sc = StandardScaler()
Xr_tr = sc.fit_transform(X_tr[:, :max_len])
Xr_te = sc.transform(X_te[:, :max_len])
```

Truncates each time-series to 50 steps (or shorter if the dataset is shorter), then standardizes. The scaler is fit on training data only and applied to test data — this prevents data leakage.

```
clf_raw = GradientBoostingClassifier(n_estimators=100, max_depth=3,
random_state=42)
clf_raw.fit(Xr_tr, y_tr)
acc_raw = accuracy_score(y_te, clf_raw.predict(Xr_te))
f1_raw  = f1_score(y_te, clf_raw.predict(Xr_te), average='weighted')
```

Trains a GradientBoosting classifier with 100 trees, depth 3, fixed random seed. Computes both accuracy and weighted F1 on the test set.

6.4 Catch22 Pipeline

```
X_c22,    feat_names = extract_catch22(X_tr)
X_c22_te, _          = extract_catch22(X_te)
sc2 = StandardScaler()
Xc_tr = sc2.fit_transform(X_c22)
Xc_te = sc2.transform(X_c22_te)
clf_c22 = GradientBoostingClassifier(n_estimators=100, max_depth=3,
random_state=42)
clf_c22.fit(Xc_tr, y_tr)
acc_c22 = accuracy_score(y_te, clf_c22.predict(Xc_te))
f1_c22  = f1_score(y_te, clf_c22.predict(Xc_te), average='weighted')
```

Same pipeline but using Catch22 features instead of raw values. The 22-dimensional feature matrix is much smaller than the 150-dimensional raw matrix.

6.5 SHAP-Pruned Pipeline

```
Xp_tr, Xp_te, kept, shap_imp, _ = shap_prune(
    Xc_tr, y_tr, Xc_te, feat_names, keep_frac=0.6, task='clf')
clf_shap = GradientBoostingClassifier(n_estimators=100, max_depth=3,
random_state=42)
clf_shap.fit(Xp_tr, y_tr)
acc_shap = accuracy_score(y_te, clf_shap.predict(Xp_te))
f1_shap  = f1_score(y_te, clf_shap.predict(Xp_te), average='weighted')
```

Calls our SHAP pruning helper to keep only the top 60% of Catch22 features. Then trains a fresh classifier on the reduced feature set.

6.6 Storing Results

```
clf_results.append({
    'dataset': ds_name,
    'raw_acc': acc_raw,   'raw_f1': f1_raw,
    'c22_acc': acc_c22,   'c22_f1': f1_c22,
    'shap_acc': acc_shap, 'shap_f1': f1_shap,
    'feat_names': feat_names,
    'shap_importance': shap_imp.tolist(),
    'n_pruned': n_pruned,
})
```

Saves all results plus the SHAP importance array (used later for the importance chart).

---

7. Cell 6 — Classification Accuracy Bar Chart

Builds a grouped bar chart with three bars per dataset (Raw, Catch22, SHAP). Each bar is annotated with its accuracy value. The bottom of the cell prints a clean summary table showing which method wins on each dataset.

---

8. Cell 7 — SHAP Feature Importance Chart (GunPoint)

8.1 Sort and Plot

```
r = clf_results[0]   # GunPoint is the first dataset
feat_names  = r['feat_names']
shap_imp    = np.array(r['shap_importance'])
order       = np.argsort(shap_imp)[::-1]
sorted_names = [feat_names[i] for i in order]
sorted_vals  = shap_imp[order]
```

Pulls the GunPoint result, converts the importance list back to an array, and sorts features by importance in descending order.

```
fig, ax = plt.subplots(figsize=(8, 6))
colors = plt.cm.RdYlGn(np.linspace(0.15, 0.9, len(sorted_names)))[::-1]
ax.barh(range(len(sorted_names)), sorted_vals[::-1], color=colors[::-1])
```

Creates a horizontal bar chart with a red-yellow-green gradient. Top features are green, bottom features are red. The `[::-1]` reversals are because `barh` plots from the bottom up.

---

9. Cell 8 — PCA Projection (SyntheticControl)

```
X_tr_sc, y_tr_sc, _, _ = ucr_data['SyntheticControl']
X_c22_sc, _ = extract_catch22(X_tr_sc)
X_sc_scaled = StandardScaler().fit_transform(X_c22_sc)
pca = PCA(n_components=2, random_state=42)
X_2d = pca.fit_transform(X_sc_scaled)
```

Extracts the 22-dimensional Catch22 features for SyntheticControl, standardizes them, then projects to 2D using PCA. The PCA finds the two directions that capture the most variance.

```
class_names = ['Normal', 'Cyclic', 'Increasing', 'Decreasing', 'Upward
Shift', 'Downward Shift']
palette = sns.color_palette('tab10', n_colors=6)

fig, ax = plt.subplots(figsize=(8, 5.5))
for cls in np.unique(y_tr_sc):
    mask  = y_tr_sc == cls
    ax.scatter(X_2d[mask, 0], X_2d[mask, 1],
               color=palette[cls], label=class_names[cls], s=50, alpha=0.8)
```

Plots each class with a different color. The result shows that the six classes form distinct clusters in 2D Catch22 space, which explains why classification works so well.

---

10. Cell 9 — Build Lag and Rolling Features for ETTh1

10.1 Helper Function

```
def build_lag_rolling(series, lags=(1, 2, 3, 6, 12, 24), windows=(3, 6, 12,
24)):
    df = pd.DataFrame({'value': series})
    for lag in lags:
        df[f'lag_{lag}'] = df['value'].shift(lag)
    for w in windows:
        roll = df['value'].rolling(window=w)
        df[f'roll_mean_{w}'] = roll.mean()
        df[f'roll_std_{w}']  = roll.std()
        df[f'roll_min_{w}']  = roll.min()
        df[f'roll_max_{w}']  = roll.max()
    df.dropna(inplace=True)
    return df
```

For each of the 6 lags, creates a column with the value shifted by that many time steps. For each of the 4 windows, creates four rolling statistics (mean, std, min, max). After computing, drops rows with NaN values (the first 24 rows because of the longest lag).

10.2 Building the Feature Matrix

```
df_ett = ett_df.head(8000).copy().reset_index(drop=True)
target_col   = 'OT'
feature_cols = [c for c in df_ett.columns if c not in ['date', target_col]]

feat_df = build_lag_rolling(df_ett[target_col])
aligned = df_ett.loc[feat_df.index].copy()
for col in feature_cols:
    feat_df[col] = aligned[col].values
```

Uses the first 8000 rows of ETTh1 for speed. Builds lag/rolling features for the OT column, then adds the current values of the other 6 sensor variables (HUFL, HULL, MUFL, MULL, LUFL, LULL).

```
X_full = feat_df.drop(columns=['value']).values
y_full = feat_df['value'].values
split  = int(len(X_full) * 0.8)
X_tr_f = X_full[:split]; X_te_f = X_full[split:]
y_tr_f = y_full[:split]; y_te_f = y_full[split:]
sc_f   = StandardScaler()
X_tr_s = sc_f.fit_transform(X_tr_f)
X_te_s = sc_f.transform(X_te_f)
```

Splits $^{80}/_{20}$ by time order (no shuffling — that would leak future information). Standardizes features using train-set statistics only.

---

11. Cell 10 — Run All Forecasting Models

11.1 Lag-1 Baseline

```
lag1_idx   = feat_col_names.index('lag_1')
reg_raw    = Ridge(alpha=1.0)
reg_raw.fit(X_tr_s[:, [lag1_idx]], y_tr_f)
y_pred_raw = reg_raw.predict(X_te_s[:, [lag1_idx]])
mse_raw    = mean_squared_error(y_te_f, y_pred_raw)
mae_raw    = mean_absolute_error(y_te_f, y_pred_raw)
```

Trains Ridge regression using only the lag-1 column. This is the simplest possible forecasting baseline.

11.2 Full Lag+Rolling with GradientBoosting

```
reg_lr    = GradientBoostingRegressor(n_estimators=200, max_depth=4,
random_state=42)
reg_lr.fit(X_tr_s, y_tr_f)
y_pred_lr = reg_lr.predict(X_te_s)
```

Uses all 28 lag/rolling features with a slightly larger GradientBoosting model.

11.3 Self-Supervised Embeddings

```
def get_embeddings(series, window=24, hidden=32):
    arr = np.asarray(series, dtype=float)
    X_w, y_w = [], []
    for i in range(window, len(arr)):
        X_w.append(arr[i-window:i]); y_w.append(arr[i])
    X_w = np.array(X_w); y_w = np.array(y_w)
    sc  = StandardScaler()
    X_s = sc.fit_transform(X_w)
    mlp = MLPRegressor(hidden_layer_sizes=(hidden,), max_iter=80,
                       random_state=42, early_stopping=True,
n_iter_no_change=10)
    mlp.fit(X_s, y_w)
    return np.tanh(X_s @ mlp.coefs_[0] + mlp.intercepts_[0])
```

Self-supervised embedding learning:

1. Build sliding windows: each input is 24 consecutive values, target is the next value.
2. Train an MLP with 32 hidden units to predict the next value.
3. Extract the hidden layer activations as features. The `np.tanh(X_s @ coefs[0] + intercepts[0])` manually computes the first-layer activations using the trained weights.

11.4 SHAP Pruning for Forecasting

```
Xp_tr_f, Xp_te_f, kept_f, shap_imp_f, _ = shap_prune(
    X_tr_s, y_tr_f, X_te_s, feat_col_names, keep_frac=0.6, task='reg')
```

Same pruning approach as classification, but using `task='reg'` so it uses GradientBoostingRegressor as the proxy. Removes 12 of 28 features.

---

12. Cell 11 — Forecasting Plots

Two charts side by side. The first shows MSE and MAE bar charts comparing all four methods. The second shows actual vs predicted oil temperature for the first 200 test points, overlaying the ground truth with predictions from the Lag+Rolling and Embedding models.

---

13. Cell 12 — Anomaly Detection

13.1 Sliding Windows

```
window_size = 50
step        = 10
X_wins, y_wins = [], []
for start in range(0, len(bvp) - window_size, step):
    X_wins.append(bvp[start:start + window_size])
    y_wins.append(int(labels[start:start + window_size].mean() > 0.3))
```

Slides a 50-sample window over the BVP signal with a step of 10. A window is labeled as anomalous if more than 30% of its samples are anomalies.

13.2 Three Detection Methods

```
iso_raw = IsolationForest(contamination=float(y_wins.mean()),
random_state=42)
iso_raw.fit(X_wins)
scores_raw = -iso_raw.score_samples(X_wins)
thresh_raw = np.percentile(scores_raw, 100 * (1 - y_wins.mean()))
y_pred_raw_a = (scores_raw > thresh_raw).astype(int)
```

Isolation Forest on raw windows. The contamination parameter is set to the actual anomaly rate. We negate the scores (Isolation Forest returns lower values for anomalies). Threshold is set so the predicted anomaly rate matches the true rate.

The Catch22 and SHAP-selected variants follow the same pattern but use the Catch22-transformed features instead of raw values.

---

14. Cell 13 — Anomaly Detection Charts

Two-panel chart showing F1-Score and ROC-AUC for all three methods. Below the chart, prints the SHAP feature importances so we can see which features the algorithm decided were most useful for anomaly detection.

---

15. Cell 14 — Final Results Summary

Prints a clean text-based summary table with all results from all three tasks, plus the five key takeaways from the project.

---

16. Reproducibility Notes

- Every random seed is fixed (42 for sklearn, 7 for the PPG signal generator).
- Train/test splits are deterministic.
- All datasets are downloaded from official sources.
- Running the notebook a second time will produce identical numbers.

GitHub: https://github.com/Ash-projects-personal/CSCE5222-Group18-FeatureEngineering CSCE 5222 Feature Engineering — Spring 2026 — Group 18